Security Project Report - A2 Trojan in the LLM era: Is LLM a plus or a minus in dealing with A2?

Amber Tsao (cctsao), Jiahao Xu (e1583425)

Introduction

Inspired by the brief survey of recent updates on how researchers tackle analog hardware trojans using LLMs, we are interested in gaining a deeper understanding of how these are done and how things might change as LLMs improve. We focus on two papers, including SPICED: Syntactical Bug and Trojan Pattern Identification in A/MS Circuits using LLM-Enhanced Detection (Chaudhuri et al., 2024) and LATENT: LLM-Augmented Trojan Insertion and Evaluation Framework for Analog Netlist Topologies (Chaudhuri et al., 2025), which respectively represent the defending and attacking LLMs of the analog hardware trojans issue.

SPICED feedback was originally designed for designers to not only detect trojans but also identify weaknesses and improve the design. However, such feedback may also help the attacker side to hide the trojan even better. According to LATENT's report, the average evasion rate can be improved to 94.4% with feedback from 23.7% for one-shot prompting. This leads us to question whether LLMs can be a solution to A2 Trojans or mainly leak additional clues to the attacker.

LATENT's percentages measure inserted components escaping localization, rather than whole circuits escaping detection. Our experiment holds the model fixed and compares feedback conditions.

Methodology

Our experiment consists of two small circuits as benchmarks, an LLM attacker inspired by LATENT's feedback loop, and a netlist-only LLM detector inspired by SPICED. Both use **gpt-4o-mini-2024-07-18** at temperature **0.3**. These are simplified implementations, not full reproductions. We omit syntax-repair evaluation.

First, the attacker receives the clean netlist, normal and activating test conditions, and a menu of permitted changes. It returns four choices: an insertion site, a device representation, a trigger profile, and a loading strength. For example, an AND proposal can select a MOSFET branch from the output to ground, with trigger profile p2 and medium strength. Python translates these choices into actual SPICE components while preserving the original circuit.

Next, ngspice runs the modified circuit under normal inputs and an activating burst. Separately, the detector receives only the candidate netlist and its nominal function. It returns a Boolean Trojan verdict, confidence, suspicious identifiers and an explanation. The detector sees neither the clean reference nor simulation results. We count an escape only if the circuit passes both electrical checks and the detector returns is_trojan=false. Simulator results are recorded for evaluation but are not fed to either LLM.

The experiment is conducted under 3 feedback conditions to test if different levels of detail feedback affect evasion rate. “None” means that the attacker receives no feedback from the reviewer; “Binary” means that the attacker receives only the result of whether the attack is detected or not; “Full” means that suspicious nodes and explanation being found by the detector is provided back to the attacker.

Each request produces a complete next configuration. The loop does not retain a best-so-far candidate or stop after an escape; it continues for the fixed number of attempts. Thus, feedback can lead to a useful revision, an unnecessary change, or a return to an earlier design.

Methodology (continued)

Binary and full also receive their own preceding candidate's choices; none has no history. Full includes the verdict and confidence as well as suspicious nodes and explanation. Feedback goes only to the attacker. The detector uses a fresh request with fixed prompts and weights each time; it does not learn during the loop. Its control-based prompt calibration occurred before the final attacks.

Each session contains ten attempts: one initial proposal followed by nine further requests. We run two sessions per condition in each of two execution batches, for **four sessions and 40 attempts per circuit/condition**. The batches use different predeclared seeds with the same model, prompts and circuit settings. The same initial candidate and review start all three conditions within a session. None continues with fresh requests; binary and full revise from the preceding attempt.

Circuits and permitted modifications

We use a two-stage op-amp and a four-input AND because they provide two readily checked kinds of behavior: continuous amplification and a digital truth table. Small circuits also keep simulation and manual inspection manageable. Both use 3.3 V supplies. Their clean versions are verified before any modification:

Clean circuit	Verification
Two-stage op-amp	Open-loop gain 95.47 dB; -3 dB bandwidth 131.90 Hz; unity-gain frequency 9.208 MHz; phase margin 50.38 degrees.
Four-input AND	16/16 logic cases pass. Rise delay 0.1476 ns; fall delay 0.1147 ns when all four inputs fall together (100 fF load, 100 ps edges).

The trigger works like a leaky bucket. Input transitions add charge through a coupling capacitor and two diodes to a 10 pF reservoir; a resistor slowly drains it. Rapid activity can raise the reservoir above a threshold, enabling an extra branch that pulls an output or internal node toward a supply rail. The intended pattern is ordinary behavior while dormant and an output disturbance after activation. This captures A2's charge-accumulation idea, using simulation models rather than fabricated hardware.

The attacker chooses among three op-amp sites (output, gain node or mirror-reference node) and two AND sites (final output or the NAND node feeding its inverter). Each site has three branch representations, three RC/threshold profiles and three strengths: **81 op-amp and 54 AND configurations**. The representations are a switch, smooth conductance and MOSFET; the MOSFET option uses an ideal reservoir comparator. This bounded menu makes the experiment about selecting and revising effective modifications, without mixing in arbitrary netlist syntax failures.

Electrical checks and their purpose

A candidate must pass two checks to count as an escape. **Normal behavior:** the op-amp follower stays within 20 mV of its clean reference; AND retains all 16 logic cases and stays within 0.2 V at settled outputs. **Activation:** the op-amp output falls at least 0.3 V below its reference, and AND's high output falls by at least 0.8 V. A harmless added branch or a permanently broken circuit would therefore not qualify merely because the LLM missed it.

The op-amp changes from 1.65 V +/- 50 mV at 10 kHz to a 0.6-2.7 V, 1 MHz burst at 5-45 us; the output drop is measured at absolute time 50-60 us. Both amplitude and frequency change, so frequency alone is not isolated. AND receives a 1 MHz burst on input a while b, c and d are high. All 135 allowed configurations passed the declared tests before the LLM runs, and each generated candidate was simulated again. Validity therefore measures selection from a qualified menu, not unrestricted LLM circuit-design ability.

Results

The six groups contain 240 analysis rows: 2 circuits x 3 conditions x 4 sessions x 10 attempts. Eight shared initial events each appear in three conditions, giving 16 reused rows and 224 actual candidate/review events. Repeated choices reduce these to 29 distinct netlists. All 224/224 candidate events pass the electrical tests. Two reviews are unusable because their responses were truncated; they remain non-successes in the per-attempt denominators, not positive detections. One subsequent binary request retained the preceding choices without a verdict.

An **escape** requires electrical validity and a usable negative Trojan verdict. The original outcome is escapes per attempt. To separate one request from a larger request budget, we also retrospectively summarize the saved first attempt and whether any of ten attempts escaped. These additional summaries use existing records; no new experiments were performed.

Circuit	Feedback	First attempt	Any within 10	All attempts
Two-stage op-amp	none	0/4	0/4	0/40 (0.0%)
Two-stage op-amp	binary	0/4	4/4	13/40 (32.5%)
Two-stage op-amp	full	0/4	3/4	13/40 (32.5%)
Four-input AND	none	2/4	4/4	10/40 (25.0%)
Four-input AND	binary	2/4	4/4	9/40 (22.5%)
Four-input AND	full	2/4	3/4	6/40 (15.0%)

Table 1. First-attempt and any-within-ten columns count sessions (four per group); the original all-attempt column counts requests (40 per group). Initial outcomes are shared across conditions. Repeated netlists and unsuccessful reviews are retained.

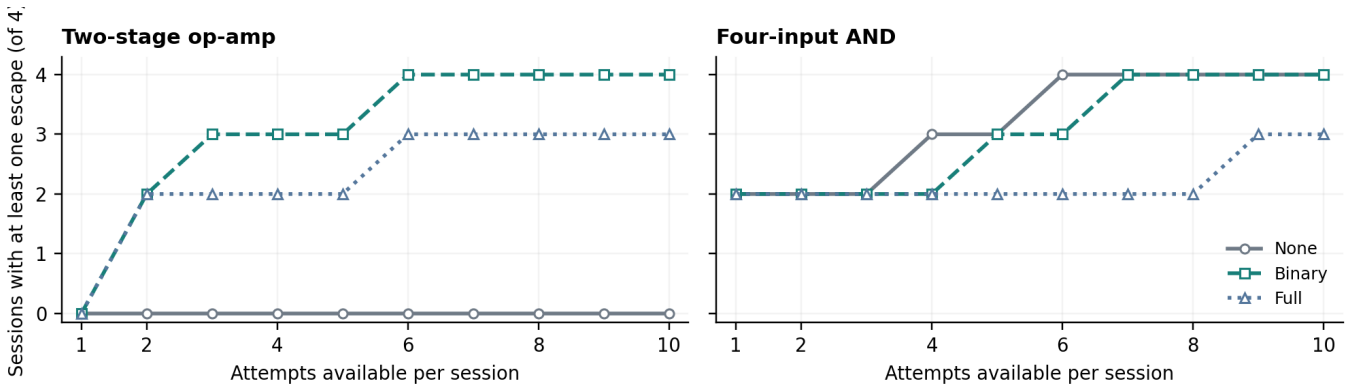


Figure 1. Retrospective cumulative opportunity to observe at least one escape. Each curve summarizes four sessions. More opportunities can reveal a miss even without a design change; this is not a measure of learning alone.

For the op-amp, both revision conditions have 13/40 escapes while none has 0/40. Binary finds an escape in all four sessions and full in three. For AND, the per-attempt order is none 10/40, binary 9/40 and full 6/40; none and binary each find an escape in all four sessions. Detailed explanation therefore does not consistently improve either measure.

Finding one escape is also different from retaining it. At attempt ten, op-amp binary and full each escape in only 1/4 sessions; AND binary and full are both 0/4. The loop continues after an escape and may replace a previously successful design. With only four sessions and dependent steps within them, these proportions are descriptive rather than evidence of statistical superiority.

Results (continued)

Consistency between execution batches

Circuit	Batch 1: none / binary / full	Batch 2: none / binary / full
Two-stage op-amp	0 / 5 / 6	0 / 8 / 7
Four-input AND	4 / 6 / 5	6 / 3 / 1

Table 2. Escape counts, each out of 20 attempts. Both batches contain two ten-attempt sessions per group; neither batch is a one-shot condition.

Full slightly exceeds binary for the op-amp in batch 1, but binary exceeds full in batch 2. AND also changes between batches. Pooling increases the number of observations; it does not remove this variation or establish a general none < binary < full ordering.

Detector controls

Two prompt refinements using recorded control results preceded every attack in the final experiment. Final controls reuse those benchmark designs and are **not held out**. They test repeated judgments of four particular circuits, not general detector accuracy. We set a gate of at most one clean flag and at least four known-positive flags in five reviews of each design. Batch 1 has 0/5 clean flags and 4/5 positive flags for both circuits.

Batch 2 again has 0/5 clean flags but only **3/5** positive flags for each circuit, failing the unchanged gate. After batch 1 results and batch 2 controls were known, an explicit continuation-rule **deviation** was recorded before batch 2 attacks. Prompts, seeds, menus and attempt counts were unchanged, and controls were not rerun. Batch 2 is therefore a same-settings stability assessment, not confirmation that passed the original gate.

Why repeat a request with no feedback?

Fresh model calls can select different configurations or give different reviews of the same configuration. The no-feedback condition gives this repeated sampling the same ten-request budget as the revision conditions. In fact, both AND no-feedback groups chose exactly the same output-to-ground MOSFET, trigger profile p2 and medium strength on every attempt. This design escaped 4/20 times in batch 1 and 6/20 in batch 2 without being modified.

The design is electrically effective: normal output differs by only 30.926 microvolts, while activation lowers its high output by 2.634 V. Across all conditions, this identical netlist is flagged in 42/60 actual reviews. A negative review describes the extra components as not altering the intended function, contrary to the simulated disturbance. Repeated misses here reflect inconsistent interpretation of the same circuit, not successive improvements to it.

The revision conditions also change which designs are reviewed. Binary and full each select the internal NAND node for 10/20 attempts in *each* batch. All 40 actual reviews of those internal-site candidates flag a Trojan; these are repeated events, not 40 distinct designs. Thus both spend half their attempts on consistently detected choices. Full does not use this site more often than binary, so that alone cannot explain their difference.

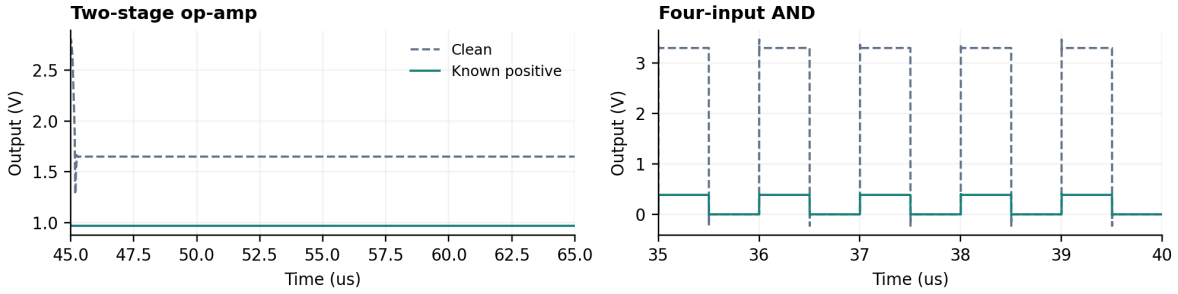


Figure 2. Clean and known-positive outputs: op-amp after the burst and AND during the burst. These traces establish electrical effects; the detector does not receive them.

Discussion

What may account for the results?

The op-amp no-feedback requests repeatedly select one configuration and produce no escape (39 flags and one unusable review). The revision conditions explore other sites and representations. Conditioning on the preceding proposal and verdict may help leave that default, but our comparison does not separate candidate history from feedback. For AND, an unchanged output-shunt design is already missed intermittently, while revisions often move to detected internal-node changes. Pulling the NAND node high directly forces the following inverter's output low; that interference may be easier to recognize. We did not isolate this mechanism experimentally. Nor did we vary circuit complexity independently of the 54-choice AND and 81-choice op-amp menus.

Why this differs from the papers

Metric and evidence. LATENT's 23.7% and 94.4% figures summarize the proportion of inserted components escaping localization in resulting designs [3, Table I]. A design can score highly on that measure while still being flagged through one detected component. Our whole-candidate verdict is stricter in this sense, and our per-attempt denominator includes all requests. SPICED gives the reviewer voltage/current logs, specifications and few-shot anomaly rules as well as the netlist [2, Sec. IV and Table III]. Our netlist-only reviewer must infer behavior from structure, which may help explain why an electrically harmful branch can be described as harmless.

Search and activation. LATENT permits broader component/node edits, uses simulation-informed diagnosis and reverts to a previous undetected configuration after detection. It reports 6-12 iterations and studies DC-activated Trojans [3, Secs. IV-V]. We use ten fixed attempts, a qualified finite menu, preceding-review feedback and no rollback, with transient charge accumulation. Our retrospective one-versus-ten comparison is therefore a request-budget summary, not a reproduction of LATENT's one-shot comparison. LATENT also uses GPT-4o-mini at temperature 0.3 for both roles; these results do not test whether a newer model improves attack or defense.

Does the LLM help the attacker or defender more?

The evidence supports a limited answer. Reviews often identify our effective modifications, but misses and inconsistent judgments prevent relying on the reviewer alone. Exposed feedback is associated with more op-amp escapes, yet detailed explanations do not outperform Boolean verdicts consistently and do not improve AND's per-attempt result. A designer could use explanations and simulation to investigate suspicious behavior; an attacker can use the same clues to choose revisions. We tested only feedback disclosure against a fixed reviewer. Without a non-LLM comparison or a defender that learns from reviews, we cannot quantify which side gains more overall.

The conclusions are bounded by four sessions per group, repeated netlists, calibrated controls and the repeat gate deviation. Electrical validity covers the declared nominal stimuli, not process variation or silicon. The main finding is that more feedback detail does not guarantee more evasion: search choices, repeated-review variation and the definition of success all matter.

References

[1] K. Yang et al. "A2: Analog Malicious Hardware." IEEE S&P, 2016, pp. 18-37. doi:10.1109/SP.2016.10.

[2] J. Chaudhuri et al. "SPICED: Syntactical Bug and Trojan Pattern Identification in A/MS Circuits using LLM-Enhanced Detection." 2024. arXiv:2408.16018v1.

[3] J. Chaudhuri, A. Chaudhuri and K. Chakrabarty. "LATENT: LLM-Augmented Trojan Insertion and Evaluation Framework for Analog Netlist Topologies." 2025. arXiv:2505.06364v1.

Code and evidence: github.com/MikeJiahaoXu/a2-llm-trojan-study. AI tools assisted implementation and writing; measurements come from saved responses and simulations.